QUERIES :

1. Consider each document in the zipcode collection has the following form:

```
{
"_id": "10280",
"city": "NEW YORK",
"state": "NY",
"pop": 5574,
}
```

Return States with Populations above 10 Million

```
db.zipcode.insertMany([
  { _id: "110001", city: "New Delhi", state: "DL", pop: 500000 },
  { _id: "400001", city: "Mumbai", state: "MH", pop: 12000000 },
  { _id: "600001", city: "Chennai", state: "TN", pop: 7000000 },
  { _id: "700001", city: "Kolkata", state: "WB", pop: 8000000 },
  { _id: "560001", city: "Bengaluru", state: "KA", pop: 9000000 },
  { _id: "500001", city: "Hyderabad", state: "TG", pop: 6800000 }
])
```

```
> db.zipcode.insertMany([
...   { _id: "110001", city: "New Delhi", state: "DL", pop: 500000 },
...   { _id: "400001", city: "Mumbai", state: "MH", pop: 12000000 },
...   { _id: "600001", city: "Chennai", state: "TN", pop: 7000000 },
...   { _id: "700001", city: "Kolkata", state: "WB", pop: 8000000 },
...   { _id: "560001", city: "Bengaluru", state: "KA", pop: 9000000 },
...   { _id: "500001", city: "Hyderabad", state: "TG", pop: 6800000 }
... ])
{
        "acknowledged" : true,
        "insertedIds" : [
                "110001",
                "400001",
                "600001",
                "700001",
                "560001",
                "500001"
        ]
}
```

```
db.zipcode.aggregate([
  { $group: { _id: "$state", totalPop: { $sum: "$pop" } } },
  { $match: { totalPop: { $gt: 10000000 } } }
])
```

```
> db.zipcode.aggregate([
...   { $group: { _id: "$state", totalPop: { $sum: "$pop" } } },
...   { $match: { totalPop: { $gt: 10000000 } } }
... ])
{ "_id" : "KA", "totalPop" : 13500000 }
{ "_id" : "WB", "totalPop" : 12200000 }
{ "_id" : "MH", "totalPop" : 20000000 }
{ "_id" : "TN", "totalPop" : 11000000 }
```

2. Display the department wise average salary

```
db.employee.insertMany([
  { emp_id: 101, name: "Rajesh Kumar", department: "IT", salary: 25000 },
  { emp_id: 102, name: "Priya Singh", department: "HR", salary: 30000 },
  { emp_id: 103, name: "Anil Sharma", department: "IT", salary: 35000 },
  { emp_id: 104, name: "Sanya Reddy", department: "Finance", salary: 40000 },
  { emp_id: 105, name: "Vikram Jain", department: "HR", salary: 32000 },
  { emp_id: 106, name: "Meera Patel", department: "Finance", salary: 45000 },
  { emp_id: 107, name: "Rahul Verma", department: "IT", salary: 50000 }
])
```

```
> db.employee.insertMany([
...   { emp_id: 101, name: "Rajesh Kumar", department: "IT", salary: 25000 },
...   { emp_id: 102, name: "Priya Singh", department: "HR", salary: 30000 },
...   { emp_id: 103, name: "Anil Sharma", department: "IT", salary: 35000 },
...   { emp_id: 104, name: "Sanya Reddy", department: "Finance", salary: 40000 },
...   { emp_id: 105, name: "Vikram Jain", department: "HR", salary: 32000 },
...   { emp_id: 106, name: "Meera Patel", department: "Finance", salary: 45000 },
...   { emp_id: 107, name: "Rahul Verma", department: "IT", salary: 50000 }
... ])
{
        "acknowledged" : true,
        "insertedIds" : [
                ObjectId("69d62c3877679ace877081a3"),
                ObjectId("69d62c3877679ace877081a4"),
                ObjectId("69d62c3877679ace877081a5"),
                ObjectId("69d62c3877679ace877081a6"),
                ObjectId("69d62c3877679ace877081a7"),
                ObjectId("69d62c3877679ace877081a8"),
                ObjectId("69d62c3877679ace877081a9")
        ]
}
```

db.employee.aggregate([
  { $group: { _id: "$department", avgSalary: { $avg: "$salary" } } }
])

```
> db.employee.aggregate([
...   { $group: { _id: "$department", avgSalary: { $avg: "$salary" } } }
... ])
{ "_id" : "Finance", "avgSalary" : 41666.666666666664 }
{ "_id" : "IT", "avgSalary" : 33000 }
{ "_id" : "HR", "avgSalary" : 26750 }
```

3. display the no. Of employees working in each department

db.employee.aggregate([
  { $group: { _id: "$department", empCount: { $sum: 1 } } }
])

```
> db.employee.aggregate([
...   { $group: { _id: "$department", empCount: { $sum: 1 } } }
... ])
{ "_id" : "Finance", "empCount" : 3 }
{ "_id" : "IT", "empCount" : 5 }
{ "_id" : "HR", "empCount" : 4 }
```

4. Display the department wise total salary of departments having total salary greater than or equals to 50000/-

```
db.employee.aggregate([
  { $group: { _id: "$department", totalSalary: { $sum: "$salary" } } },
  { $match: { totalSalary: { $gte: 50000 } } }
])
```

```
> db.employee.aggregate([
...   { $group: { _id: "$department", totalSalary: { $sum: "$salary" } } },
...   { $match: { totalSalary: { $gte: 50000 } } }
... ])
{ "_id" : "Finance", "totalSalary" : 125000 }
{ "_id" : "IT", "totalSalary" : 165000 }
{ "_id" : "HR", "totalSalary" : 107000 }
```

5. Write the queries using the different operators like max, min. Etc.

```
db.employee.aggregate([
  { $group: {
      _id: "$department",
      maxSalary: { $max: "$salary" },
      minSalary: { $min: "$salary" }
  }}
])
```

```
> db.employee.aggregate([
...   { $group: {
...       _id: "$department",
...       maxSalary: { $max: "$salary" },
...       minSalary: { $min: "$salary" }
...     }
...   }
... ])
{ "_id" : "Finance", "maxSalary" : 45000, "minSalary" : 40000 }
{ "_id" : "IT", "maxSalary" : 50000, "minSalary" : 25000 }
{ "_id" : "HR", "maxSalary" : 32000, "minSalary" : 20000 }
```

6. Create the simple index on roll_no field

```
db.student.insertMany([
  { roll_no: 1, name: "Arjun", department: "CSE", email: "arjun@pccoe.edu.in" },
  { roll_no: 2, name: "Sneha", department: "IT", email: "sneha@pccoe.edu.in" },
  { roll_no: 3, name: "Rohan", department: "ECE", email: "rohan@pccoe.edu.in" },
  { roll_no: 4, name: "Priya", department: "CSE", email: "priya@pccoe.edu.in" },
  { roll_no: 5, name: "Karthik", department: "IT", email: "karthik@pccoe.edu.in" }
])
```

```
> db.student.insertMany([
...   { roll_no: 1, name: "Arjun", department: "CSE", email: "arjun@pccoe.edu.in" },
...   { roll_no: 2, name: "Sneha", department: "IT", email: "sneha@pccoe.edu.in" },
...   { roll_no: 3, name: "Rohan", department: "ECE", email: "rohan@pccoe.edu.in" },
...   { roll_no: 4, name: "Priya", department: "CSE", email: "priya@pccoe.edu.in" },
...   { roll_no: 5, name: "Karthik", department: "IT", email: "karthik@pccoe.edu.in" }
... ])
{
        "acknowledged" : true,
        "insertedIds" : [
                ObjectId("69d62c2777679ace8770819e"),
                ObjectId("69d62c2777679ace8770819f"),
                ObjectId("69d62c2777679ace877081a0"),
                ObjectId("69d62c2777679ace877081a1"),
                ObjectId("69d62c2777679ace877081a2")
        ]
}
```

```
db.student.createIndex({ roll_no: 1 })
```

```
> db.student.createIndex({ roll_no: 1 })
{
        "createdCollectionAutomatically" : false,
        "numIndexesBefore" : 1,
        "numIndexesAfter" : 2,
        "ok" : 1
}
```

7. create unique index on any field for above given collections

```
db.student.createIndex({ email: 1 }, { unique: true })
```

```
> db.student.createIndex({ email: 1 }, { unique: true })
{
        "createdCollectionAutomatically" : false,
        "numIndexesBefore" : 2,
        "numIndexesAfter" : 3,
        "ok" : 1
}
```

8. create compound index on any fields for above given collections

db.student.createIndex({ department: 1, roll_no: -1 })

```
> db.student.createIndex({ department: 1, roll_no: -1 })
{
        "createdCollectionAutomatically" : false,
        "numIndexesBefore" : 3,
        "numIndexesAfter" : 4,
        "ok" : 1
}
```

9. Show all the indexes created in the database PCCOE

```
> db.student.getIndexes()
[ ]
```

```
> db.zipcode.getIndexes()
[ ]
```

```
> db.employee.getIndexes()
[ ]
```

10. Show all the indexes created in above collections.

db.getCollectionNames().forEach(function(c){print(c);
db[c].getIndexes().forEach(printjson); })

```
> db.getCollectionNames().forEach(function(c){ print(c); db[c].getIndexes().forEach(printjson); })
Department
{ "v" : 2, "key" : { "_id" : 1 }, "name" : "_id_" }
Students
{ "v" : 2, "key" : { "_id" : 1 }, "name" : "_id_" }
Teachers
{ "v" : 2, "key" : { "_id" : 1 }, "name" : "_id_" }
```